

Empirical Analysis of Sorting & Searching Algorithms

Hardware-level benchmarking of 10 fundamental algorithms — theory vs measured performance

Course: Design & Analysis of Algorithms · Supervised by Dr. Bashir Hayat

10

Algorithms benchmarked

$n \rightarrow 5M$

Input sizes swept

2,759×

Quick vs Bubble @ $n=10,000$

131

Raw benchmark CSV runs

Introduction

We measure how sorting and searching algorithms behave in practice versus their textbook complexity. Seven sorts (Bubble, Selection, Insertion, Shell, Heap, Merge, Quick) and three searches (Linear, Binary, Interpolation) were timed on identical hardware across geometrically spaced input sizes, using CPU-time and memory instrumentation.

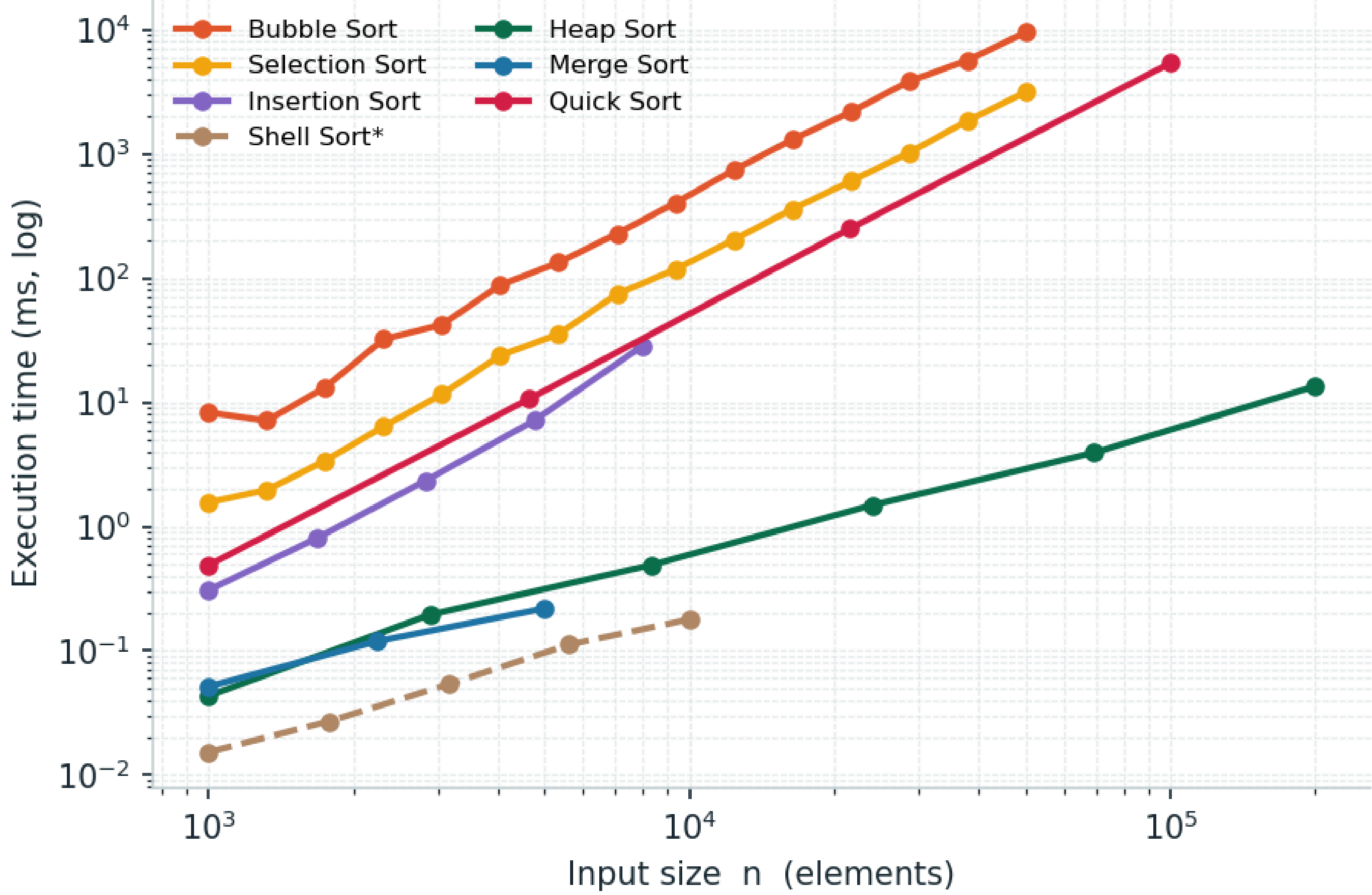
Result: asymptotic class reliably predicts *scaling*, but constant factors and data layout decide the *real* winner. $O(n \log n)$ sorts pull decisively ahead past a few thousand elements; among searches, interpolation and binary stay flat while linear grows with n .

Complexity Reference

Algorithm	Best	Average	Worst	Space	Stable
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Shell Sort	$O(n \log n)$	$O(n^{1.3})$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	—
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	—
Interp. Search	$O(1)$	$O(\log \log n)$	$O(n)$	$O(1)$	—

Sorting Performance

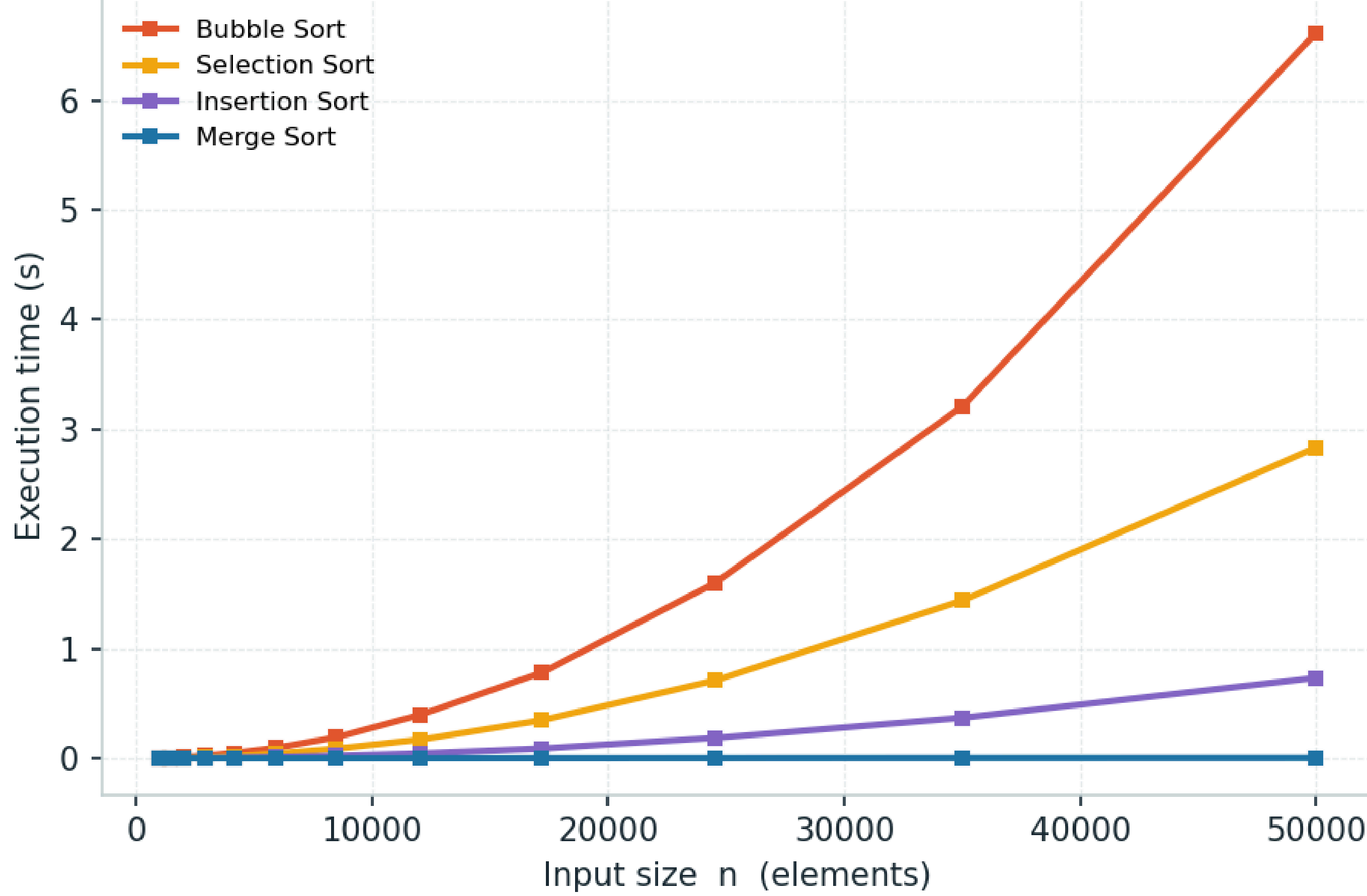
Sorting Algorithms — Full Sweep (log-log)



Each line runs to its full sweep size — Heap to $n=200k$, Quick to $100k$, Bubble/Selection to $50k$. Straight log-log lines confirm power-law scaling; quadratic sorts share a steeper slope than the $n \log n$ group. (* = no sweep file, shown over comparison range $n \leq 10k$.)

Scaling to 50k

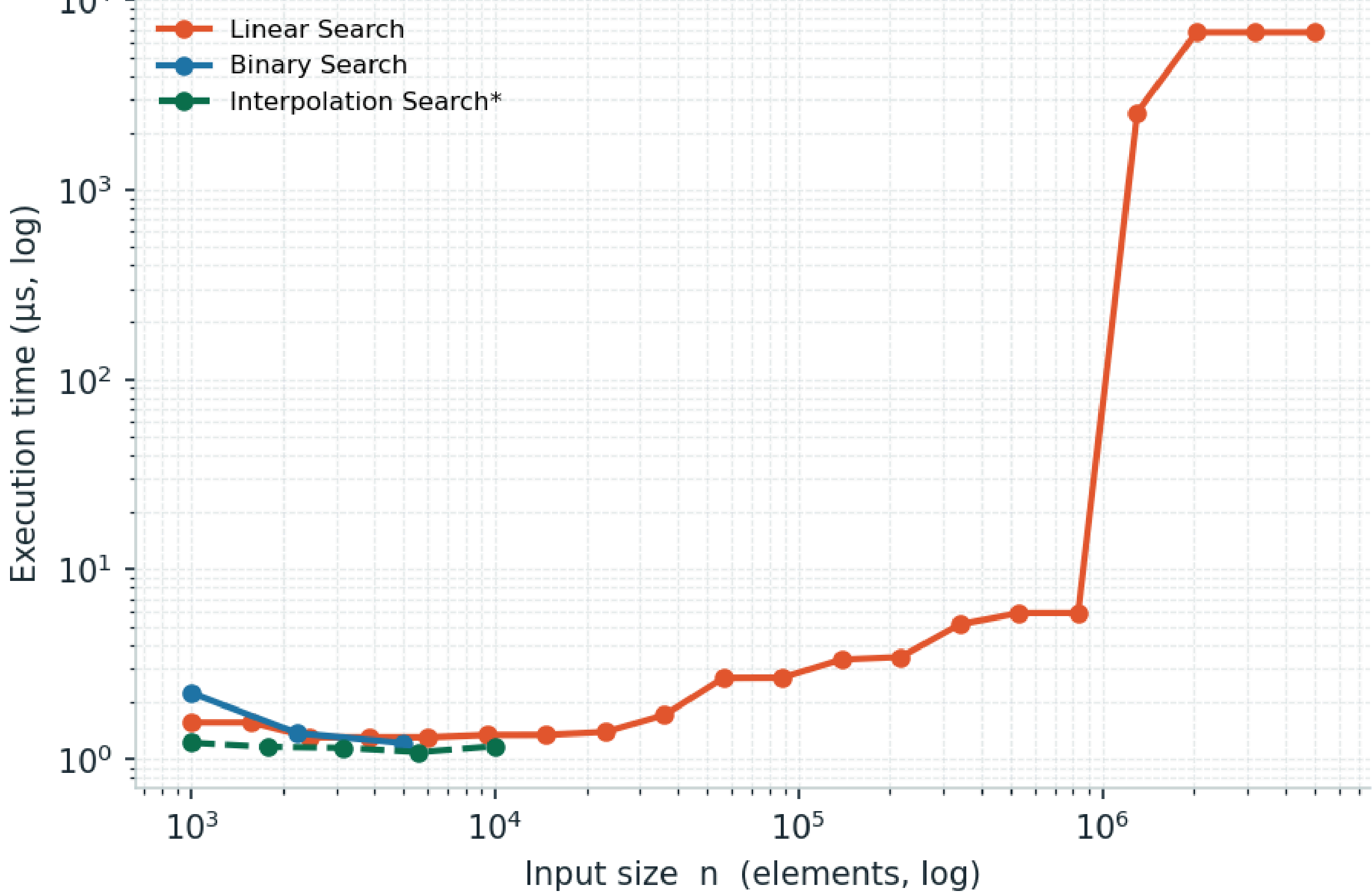
Quadratic vs Linearithmic Scaling ($n \rightarrow 50k$)



On a linear axis the $O(n^2)$ cost explodes: Bubble Sort hits 6.62s at $n=50k$ while Merge Sort stays at 2.7ms.

Searching Performance

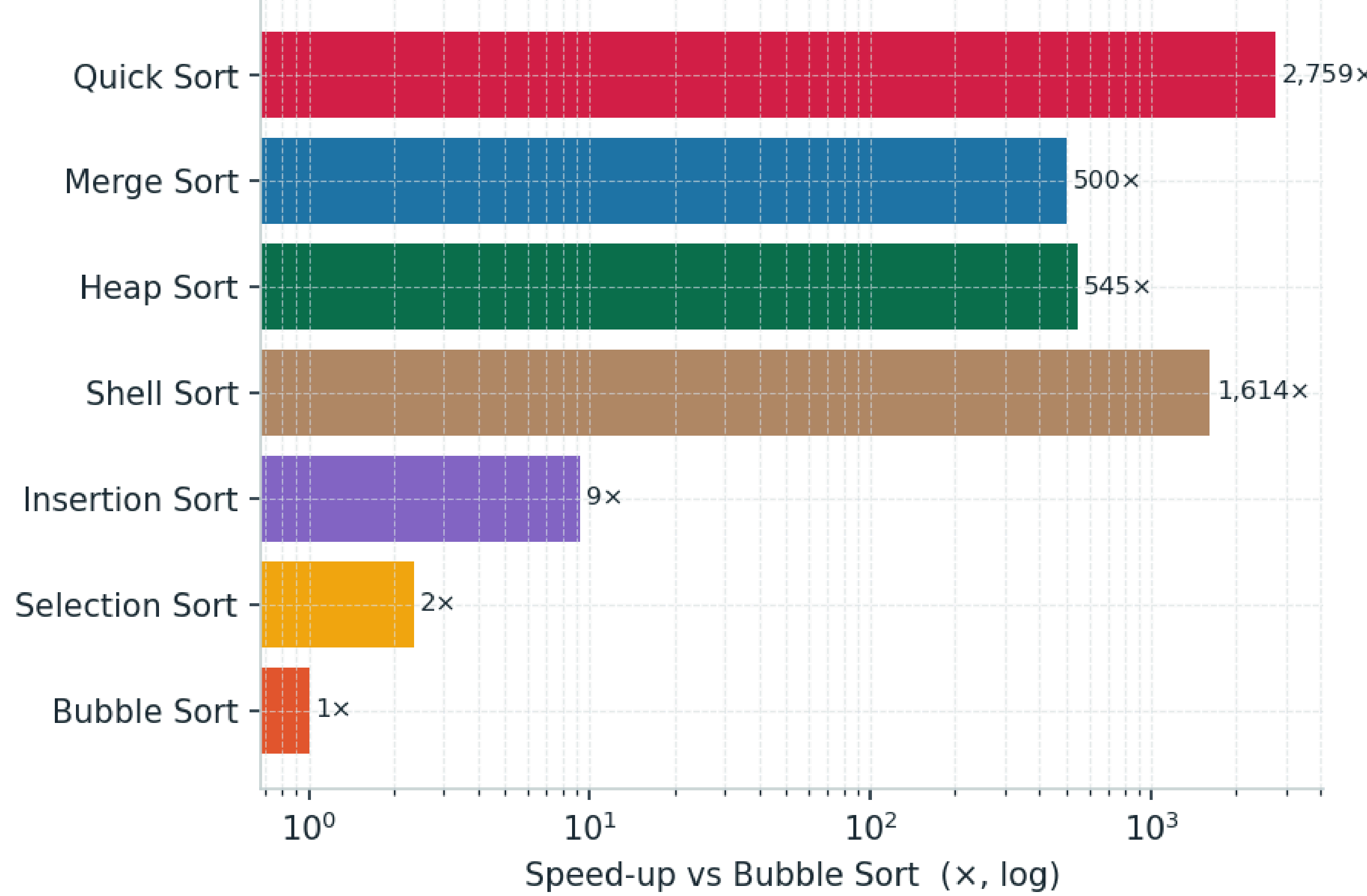
Searching Algorithms — Full Sweep (log-log)



Linear search swept to $n=5M$ rises along $O(n)$ (5-point median filtered — single-shot wall-clock outliers removed); binary stays flat in the μs range ($O(\log n)$). (* = comparison range $n \leq 10k$.)

Relative Speed-up

Relative Speed-up at $n = 10000$



Measured speed-up of each sort over Bubble Sort at the largest tested size.

Measured Times & Findings

Algorithm	t @ $n=1,000$ (ms)	t @ $n=10,000$ (ms)
Bubble Sort	3.1240	289.0788
Selection Sort	1.1234	121.7032
Insertion Sort	0.2904	31.3779
Shell Sort	0.0153	0.1791
Heap Sort	0.0747	0.5308
Merge Sort	0.0469	0.5780
Quick Sort	0.0269	0.1048
Linear Search	0.0014	0.0012
Binary Search	0.0031	0.0017
Interpolation Search	0.0012	0.0012

- Theory holds for scaling.** Every algorithm's empirical slope matches its asymptotic class.
- Constants matter.** Heap and Shell share Quick/Merge's class but run slower from larger constant factors and cache behaviour.
- Quick Sort wins in practice** on random data — best constant factors despite $O(n^2)$ worst case.
- Insertion Sort is great when small/near-sorted** — its $O(n)$ best case beats the $n \log n$ group at tiny n .
- Search:** binary/interpolation flat vs linear's growth; interpolation edges binary on uniform data.
- Memory:** Merge Sort's $O(n)$ buffer is the only notable allocator; all others stay near-constant space.